

Applied Deep Learning

Chapter 2: Feedforward Neural Networks

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

Binary Classification via FNN

Let's now design a deep FNN for *binary classification*

We have a set of *images* of *hand-written numbers*, something like this^a



We intend to train a fully-connected FNN that given a new hand-written image, *it finds out whether it is “2” or not*

^aSource: Wikipedia

This is a *binary classification*!

Binary Classification via FNN: *Data*

Let's get clear about the data: *our dataset looks like*

$$\mathbb{D} = \{(\mathbf{x}_b, v_b) \text{ for } b = 1, \dots, B\}$$

where in this set each component is defined as follows:

- B is the number of **images** we have
 - ↳ we also call the **set** of images: a **batch** of images
- $\mathbf{x}_b \in \mathbb{R}^N$ is the *pixel vector* of image b
 - ↳ N is the number of pixels in the image
- $v_b \in \{0, 1\}$ is a binary label indicating *whether it is "2" or not*
 - ↳ if the image is a hand-written "2" we set $v_b = 1$
 - ↳ if the image is **not** a hand-written "2" we set $v_b = 0$

Binary Classification via FNN: Model

Let's now set the **model**: we use a *fully-connected FNN*

What are the **hyperparameters**?

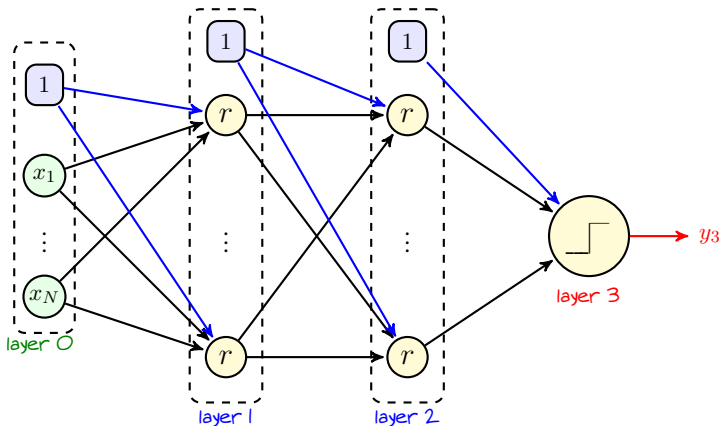
- We want it to be **deep**; so, we consider **2 hidden layers**
 ↳ the **depth** is hence **3**
- We specify the width of each hidden layer
- All hidden neurons use **ReLU activation**
 ↳ $f_1(\cdot) = f_2(\cdot) = \text{ReLU}(\cdot)$: let's **show ReLU by r** , i.e.,

$$r(x) = \text{ReLU}(x)$$

- **Output layer** has a single **perceptron**

We can now write down the model!

Binary Classification via FNN: Model



Binary Classification via FNN: Loss

How to calculate the loss? *Let's do what we did before*

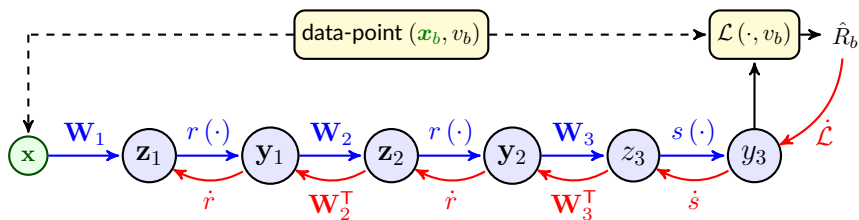
We use the error indicator as the loss function

$$\mathcal{L}(y_b, v_b) = \mathbb{1}\{y_b \neq v_b\} = \begin{cases} 1 & y_b \neq v_b \\ 0 & y_b = v_b \end{cases}$$

- + *Wait a moment! Didn't you say that this was a **bad choice**?*
- Yeah! So said I also for the **perceptron's** activation! Let's try it out to find out really **why they are bad**! We should be able to understand it now

Binary Classification via FNN: Training

How does the graph look like on the backward pass?



- + Where is then the issue with **perceptron** and **indicator error**?
- $\dot{\mathcal{L}}(y, v_b)$ and $\dot{s}(z)$ are not well-defined!

In fact, the empirical risk is not a **smooth function** of the weights and biases; therefore, using **gradient descent** we do not end up with a well-trained network

- + How can we get over it?
- Well! There is a very well-established trick!

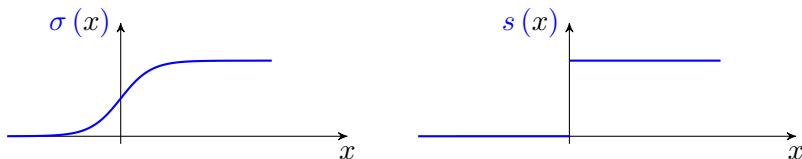
Binary Classification via FNN: *Differentiability Issue*

We first replace the **perceptron** with a neuron whose **activation** is a good approximation of *step function* and *differentiable*¹

We already have seen the **sigmoid** function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

which looks pretty close to step function



Using sigmoid instead of step function **resolves** the **differentiability** issue

¹Or at least, we can easily calculate a sub-gradient for it

Binary Classification via FNN: *Differentiability Issue*

But, replacing *perceptron* by *sigmoid-activated neuron* makes a new problem

The *output* of the network is now *not binary*!

How can we address this problem?

We now interpret the *output* as *probability*, i.e.,

y_3 is the *probability* of the *label being 1*

- + OK! But how can we define the *loss* now?
- Well! We could look at the *true label* from the same point of view

Say $v \in \{0, 1\}$ is *true label*: if $v = 1$ then the *true label is 1* with probability 1; if $v = 0$ then the *true label is 1* with probability 0. So, we could say

the *true label is 1* with probability v

Binary Classification via FNN: *Differentiability Issue*

true label is 1 with probability v $\longleftrightarrow$ y_3 is probability of the label being 1

Apparently, v and y_3 are of the same nature: we can still define a *loss* that evaluates the difference between y_3 and v

- + What should be the loss then?
- Definitely **not the indicator error!**

Indicator error is not suitable because

- ① we already know that it is **not differentiable**
- ② more importantly, with **sigmoid activation** becomes useless

$$\mathbb{1}\{\sigma(z_3) \neq 1\} = \mathbb{1}\{\sigma(z_3) \neq 0\} = 1$$

Binary Classification via FNN: MSE

One may suggest that we use the *squared error*, i.e.,

$$\mathcal{L}(y_3, v) = (y_3 - v)^2$$

in this case the empirical risk is called

Mean Squared Error (MSE)

This loss is *differentiable*

$$\dot{\mathcal{L}}(y_3, v) = 2(y_3 - v)$$

and proportional to the distance between y_3 and v

It's a good choice but not best

Binary Classification via FNN: Cross-Entropy

A better choice is to determine the **cross-entropy loss**

$$\begin{aligned}\mathcal{L}(y_3, v) &= \text{CE}(y_3, v) = -v \log y_3 - (1 - v) \log (1 - y_3) \\ &= \begin{cases} \log \frac{1}{y_3} & v = 1 \\ \log \frac{1}{1-y_3} & v = 0 \end{cases}\end{aligned}$$

This loss function is sometimes **wrongly** called **KL-divergence**: it is proportional to the **Kullback-Leibler divergence** but it's **different**

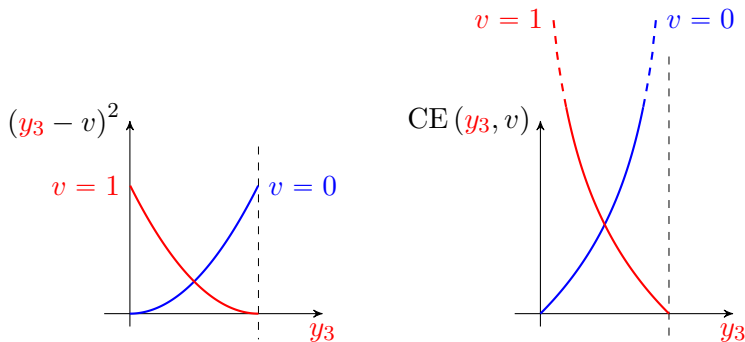
This loss is again **differentiable**

$$\dot{\mathcal{L}}(y_3, v) = \dot{\text{CE}}(y_3, v) = -\frac{v}{y_3} + \frac{1 - v}{1 - y_3}$$

Note: The logarithm is usually in natural base, i.e., $\log x = \ln x$

Binary Classification via FNN: Cross-Entropy

- + But why *cross-entropy* is a *better loss*?
- It pushes y_3 *more* towards the *edges* of interval $[0, 1]$



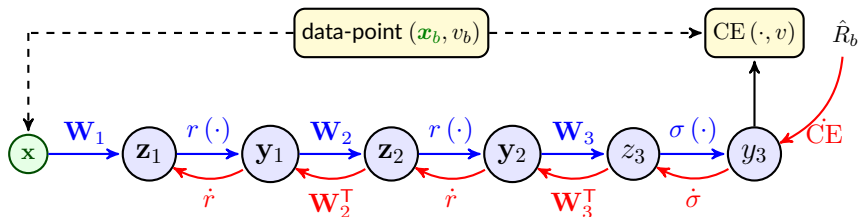
Cross entropy returns *much higher loss* when y_3 is different from v

Binary Classification via FNN: Training with Cross-Entropy

+ What changes in the training loop in this case?

- Pretty much *nothing!* Just replace

- $\mathcal{L}(y_3, v)$ with $\text{CE}(y_3, v)$
- $\dot{\mathcal{L}}(y_3, v)$ with $\dot{\text{CE}}(y_3, v)$
- $s(z_3)$ with $\sigma(z_3)$
- $\dot{s}(z_3)$ with $\dot{\sigma}(z_3)$



Binary Classification via FNN: *Inference*

- + How do we use the output of network then, when we give a new *image* to it for classification? *It's not binary!*
- Just follow the *interpretation*

y_3 gives the *probability* of the *image* being *hand-written "2"*; therefore, $(1 - y_3)$ gives the *probability* of image being *any other hand-written number*. So, we select the outcome with *higher chance*, i.e.,

- if $y_3 \geq 0.5$, we label the new *image* as a hand-written "2"
- if $y_3 < 0.5$, we label the new *image* as *not being* a hand-written "2"

- + Can't we classify more classes? Like hand-written "0", "1", ..., "9"?
- Now that we have this nice interpretation: Yes! We can!

Multiclass Classification

We initially saw that any multiclass classification can be seen as a *sequence of binary classifications*; however, for that, we need **multiple NNs**!

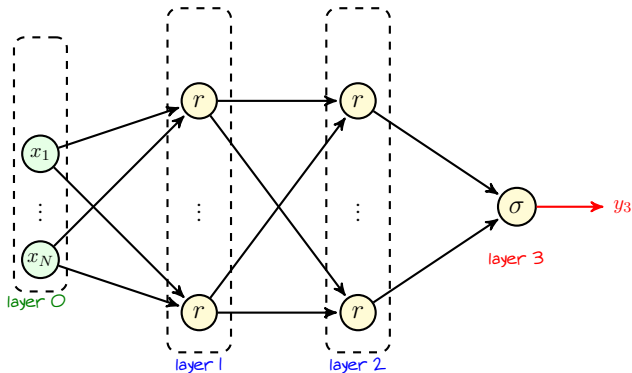
- + *Why not follow the same idea and determine the probability of input belonging to each class?*
- Yes! That's actually the effective way!

Let's get back to our *image recognition*, but now with **multiple** classes!

We have *images of hand-written numbers from "0" to "9"* and want to train a NN that *recognizes any hand-written number*

We first draw our earlier FNN

Multiclass Classification via FNN

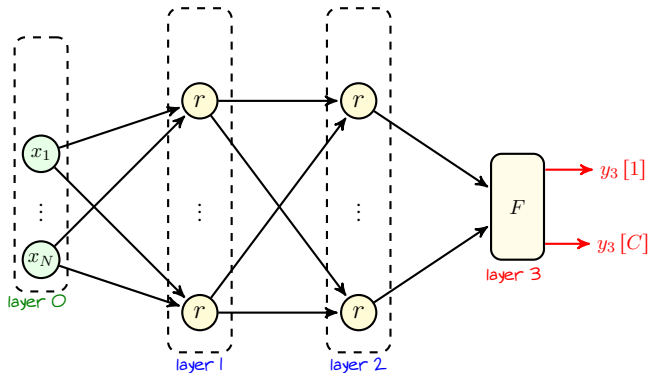


In this FNN, y_3 is interpreted as a *probability of label being 1*

probability of label being 0 is hence $1 - y_3$

This was done by a standard *single-output neuron*, since we had only *2 classes*

Multiclass Classification via FNN



With C classes, we need a module that computes probabilities of all C classes
this module can be seen as a **neuron** with **vector output**

Multiclass Classification: Vector-Activated Neuron

Vector-Activated Neuron

A **vector-activated neuron** is an artificial neuron with **multivariate activation function**: let $\mathbf{x} \in \mathbb{R}^N$ be the input to this neuron and C be its output dimension; then, the output vector $\mathbf{y} \in \mathbb{R}^C$ is given by

$$\mathbf{y} = F(\tilde{\mathbf{W}}\mathbf{x} + \mathbf{b})$$

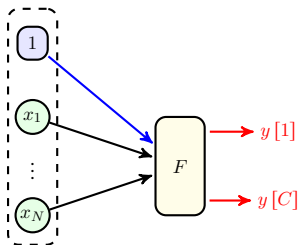
for weight matrix $\tilde{\mathbf{W}} \in \mathbb{R}^{C \times N}$, bias $\mathbf{b} \in \mathbb{R}^C$ and **activation** $F(\cdot) : \mathbb{R}^C \mapsto \mathbb{R}^C$

First thing first: let's get rid of the bias before we go on

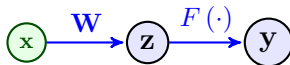
$$\mathbf{y} = F\left(\begin{bmatrix} \mathbf{b} & \tilde{\mathbf{W}} \end{bmatrix} \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix}\right) = F(\mathbf{W}\mathbf{x})$$

So, we keep on with our dummy 1 input here as well

Multiclass Classification: Vector-Activated Neuron



Next, let's see how its computation graph looks

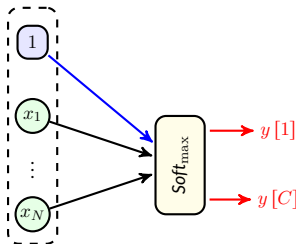


This looks exactly like a standard layer with a **minor difference**

$F(\cdot)$ does **not necessarily** perform **entry-wise**

Multiclass Classification: *Softmax*

A very well-known example of **vector activation** is *softmax*



Softmax Function

For $\mathbf{z} \in \mathbb{R}^C$, softmax function returns $\text{Soft}_{\max}(\mathbf{z}) = \mathbf{y} \in \mathbb{R}^C$ whose entry c is

$$y[c] = \frac{e^{z[c]}}{\sum_{j=1}^C e^{z[j]}}$$

Multiclass Classification: *Softmax*

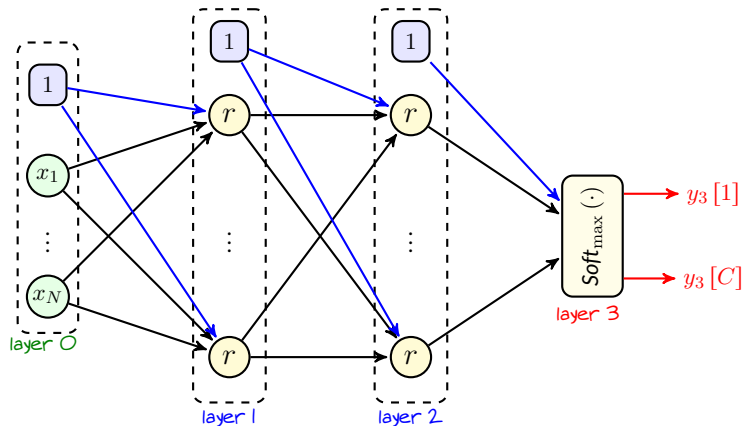
Softmax always returns a probability distribution on the set of classes

$$\sum_{c=1}^C y[c] = \sum_{c=1}^C \frac{e^{z[c]}}{\sum_{j=1}^C e^{z[j]}} = \frac{\sum_{c=1}^C e^{z[c]}}{\sum_{j=1}^C e^{z[j]}} = 1$$

We can hence use it to extend our FNN to a *multiclass classifier*

We replace *layer 3* with a *softmax-activated multivariate neuron* and treat its outcome as the *chance of input belonging to each class*; then, we select *the class with highest chance*

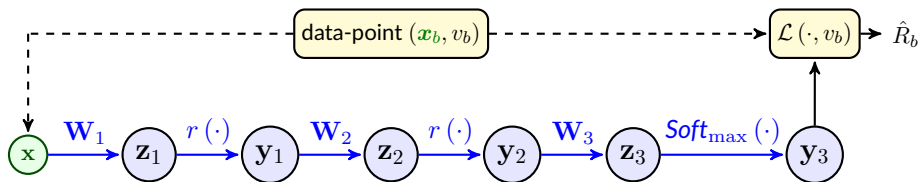
Multiclass Classification via FNN: *Softmax Activation*



Let's try again the forward pass

Multiclass Classification via FNN: *Softmax Activation*

Let's look at the computation graph: *for a given data-point $(\mathbf{x}_b, v_b)$, we have*



Note that **the output layer** has been changed

- We now have a **vector** $\mathbf{z}_3 \in \mathbb{R}^C$
 - ↳ So we have a **matrix of weights** $\mathbf{W}_3 \in \mathbb{R}^{C \times (J+1)}$
- We now have a **vector** $\mathbf{y}_3 \in \mathbb{R}^C$
- We get from $\mathbf{z}_3$ to $\mathbf{y}_3$ via **softmax**
 - ↳ This is **not** an entry-wise activation anymore!

Multiclass Classification: Loss

- + How can we define the loss now? On one side we have a vector of probabilities; on the other side an integer label!
- Again we need to convert *true labels* to *probabilities*

Let's say we have C classes: the *vector* of *probabilities* contains C entries

$$\mathbf{p} = \begin{bmatrix} p_1 \\ \vdots \\ p_C \end{bmatrix} = \begin{bmatrix} \Pr \{\text{image belongs to class } 1\} \\ \vdots \\ \Pr \{\text{image belongs to class } C\} \end{bmatrix}$$

If we know that the *image* b belongs to class v_b , we could say that

$$\mathbf{p} \text{ of image } b = \begin{bmatrix} p_1 \\ \vdots \\ p_{v_b} \\ \vdots \\ p_C \end{bmatrix} = \begin{bmatrix} \Pr \{\text{image } b \text{ belongs to class } 1\} = 0 \\ \vdots \\ \Pr \{\text{image } b \text{ belongs to class } v_b\} = 1 \\ \vdots \\ \Pr \{\text{image } b \text{ belongs to class } C\} = 0 \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

Multiclass Classification: Loss

So, we could say that *label v is corresponding to a vector of size C whose entry v is 1 and the remaining entries are all 0*: this vector is called a *one-hot vector*

One-hot Vector

The one-hot vector $\mathbf{1}_v \in \{0, 1\}^C$ is a C -dimensional vector whose entry v is 1 and all remaining entries are 0

For instance: say $C = 3$; then, we have

$$\mathbf{1}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \mathbf{1}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad \mathbf{1}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Moral of Story

*We can interpret *true label v* as a probability vector $\mathbf{1}_v$*

Multiclass Classification: Loss

We can interpret *true label* v as a probability vector $\mathbf{1}_v$

Now for *image* b with *label* v_b we compare network's output $\mathbf{y}_3$ to $\mathbf{1}_{v_b}$

$$\hat{R}_b = \mathcal{L}(\mathbf{y}_3, \mathbf{1}_{v_b})$$

for *loss* $\mathcal{L}(\cdot)$ that determines distance between two *probability vectors*

- + What kind of *loss functions* do we use usually?
- Like binary case: squared error is *good*, cross-entropy is the *best*
- + How do we define them in this case?
- Just extend them to multi-dimensional vectors

Multiclass Classification: Loss

We can extend squared error to vector form as

$$\begin{aligned}\mathcal{L}(\mathbf{y}, \mathbf{1}_v) &= \|\mathbf{y} - \mathbf{1}_v\|^2 = \sum_{c=1}^C (y[c] - \mathbb{1}\{c=v\})^2 \\ &= \sum_{c=1, c \neq v}^C y[c]^2 + (y[v] - 1)^2\end{aligned}$$

This gradient of this loss is

$$\nabla \mathcal{L}(\mathbf{y}, \mathbf{1}_v) = 2 \begin{bmatrix} y[1] \\ \vdots \\ y[v] - 1 \\ \vdots \\ y[C] \end{bmatrix} = 2(\mathbf{y} - \mathbf{1}_v)$$

Multiclass Classification: Loss

Cross entropy can also be extended as follows

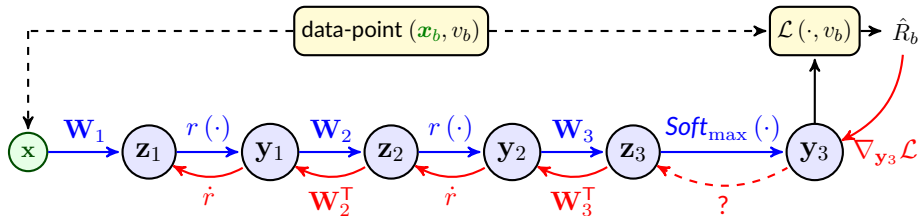
$$\begin{aligned}\mathcal{L}(\mathbf{y}, \mathbf{1}_v) &= \text{CE}(\mathbf{y}, \mathbf{1}_v) = - \sum_{c=1}^C \mathbb{1}\{c = v\} \log y(c) \\ &= -\log y[v]\end{aligned}$$

The gradient of this loss is

$$\nabla \mathcal{L}(\mathbf{y}, \mathbf{1}_v) = \nabla \text{CE}(\mathbf{y}, \mathbf{1}_v) = \begin{bmatrix} 0 \\ \vdots \\ -1/y[v] \\ \vdots \\ 0 \end{bmatrix} = -\frac{1}{y[v]} \mathbf{1}_v$$

Backpropagation Through Vector Activation

How can we backpropagate through this neural network?

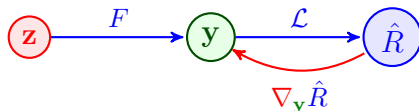


- 1 Compute $\nabla_{y_3} \hat{R}_b = \nabla_{y_3} \mathcal{L}(y_3, \mathbf{1}_{v_b})$
- 2 Compute $\nabla_{z_3} \hat{R}_b$ from $\nabla_{y_3} \hat{R}_b$ by its **backward** link that we don't know yet
- 3 Compute $\nabla_{y_2} \hat{R}_b = W_3^T \nabla_{z_3} \hat{R}_b$
- 4 Remove entry at index 0 of $\nabla_{y_2} \hat{R}_b$ and compute $\nabla_{z_2} \hat{R}_b = \dot{r}(z_2) \odot \nabla_{y_2} \hat{R}_b$
- 5 Compute $\nabla_{y_1} \hat{R}_b = W_2^T \nabla_{z_2} \hat{R}_b$
- 6 Remove entry at index 0 of $\nabla_{y_1} \hat{R}_b$ and compute $\nabla_{z_1} \hat{R}_b = \dot{r}(z_1) \odot \nabla_{y_1} \hat{R}_b$

Backpropagation Through Vector Activation

How is $\nabla_{\mathbf{z}_3} \hat{R}_b$ related to $\nabla_{\mathbf{y}_3} \hat{R}_b$? Let's do what we did before

In this graph, $F(\cdot)$ is a **vector activation**. We know $\nabla_{\mathbf{y}} \hat{R}$



We want to find $\nabla_{\mathbf{z}} \hat{R}$

As mentioned before: we can always *extend things entry-wise*

With **vector activation**, we need to use the notion of **Jacobian**

Recap: Jacobian Matrix

Consider **vector activation** $F(\cdot)$ that maps C -dimensional $\mapsto C$ -dimensional

$$\begin{bmatrix} y_1 \\ \vdots \\ y_C \end{bmatrix} = F\left(\begin{bmatrix} z_1 \\ \vdots \\ z_C \end{bmatrix}\right)$$

When we use this function, we can say

Any entry y_j is function of all^a $z_1, \dots, z_C$, so we have

$$\nabla_{\mathbf{z}} y_j = \begin{bmatrix} \partial y_j / \partial z_1 \\ \vdots \\ \partial y_j / \partial z_C \end{bmatrix}$$

^aIt is not any more an entry-wise functional operation

Recap: Jacobian Matrix

Consider **vector activation** $F(\cdot)$ that maps C -dimensional $\mapsto C$ -dimensional

$$\begin{bmatrix} y_1 \\ \vdots \\ y_C \end{bmatrix} = F\left(\begin{bmatrix} z_1 \\ \vdots \\ z_C \end{bmatrix}\right)$$

When we use this function, we can say

We can **collect all these gradients** into a **matrix**

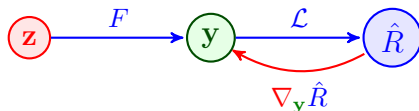
$$\mathbf{J}_{\mathbf{z}}\mathbf{y} = \mathbf{J}_{\mathbf{z}}F = \begin{bmatrix} \nabla_{\mathbf{z}} y_1^\top \\ \vdots \\ \nabla_{\mathbf{z}} y_C^\top \end{bmatrix} = \begin{bmatrix} \partial y_1 / \partial z_1 & \dots & \partial y_1 / \partial z_C \\ \vdots & & \vdots \\ \partial y_C / \partial z_1 & \dots & \partial y_C / \partial z_C \end{bmatrix}$$

and we call it the **Jacobian matrix**

Backpropagation Through Vector Activation

Now, let's get back to our problem

In this graph, $F(\cdot)$ is a **vector activation**. We know $\nabla_{\mathbf{y}} \hat{R}$



We want to find $\nabla_{\mathbf{z}} \hat{R}$: let's write down a partial derivative $\hat{R}$ w.r.t. z_c

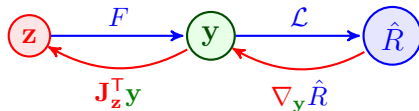
$$\frac{\partial \hat{R}}{\partial z_c} = \sum_{j=1}^C \frac{\partial \hat{R}}{\partial y_j} \frac{\partial y_j}{\partial z_c} = \underbrace{\begin{bmatrix} \frac{\partial y_1}{\partial z_c} & \cdots & \frac{\partial y_C}{\partial z_c} \end{bmatrix}}_{\text{transpose of column } c \text{ of } \mathbf{J}_{\mathbf{z}\mathbf{y}} \equiv \text{row } c \text{ of } \mathbf{J}_{\mathbf{z}\mathbf{y}}^T} \nabla_{\mathbf{y}} \hat{R}$$

Backpropagation Through Vector Activation

So, the gradient of $\hat{R}$ w.r.t. $\mathbf{z}$ is given by

$$\nabla_{\mathbf{z}} \hat{R} \begin{bmatrix} \partial \hat{R} / \partial z_1 \\ \vdots \\ \partial \hat{R} / \partial z_C \end{bmatrix} = \begin{bmatrix} \text{row 1 of } \mathbf{J}_{\mathbf{z}}^T \mathbf{y} \nabla_{\mathbf{y}} \hat{R} \\ \vdots \\ \text{row } C \text{ of } \mathbf{J}_{\mathbf{z}}^T \mathbf{y} \nabla_{\mathbf{y}} \hat{R} \end{bmatrix} = (\mathbf{J}_{\mathbf{z}}^T \mathbf{y}) \nabla_{\mathbf{y}} \hat{R}$$

So, we can complete the computation graph as follows

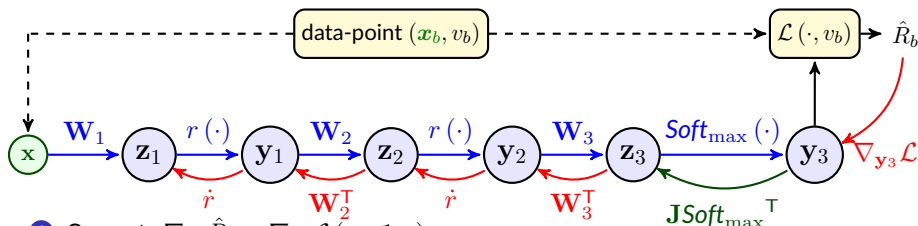


Backward Pass of Vector Activation

To pass backward on a vector activation, we use the *transpose* of its *Jacobian*

Backpropagation Through Vector Activation

How can we backpropagate through this neural network? **Let's complete**



- 1 Compute $\nabla_{y_3} \hat{R}_b = \nabla_{y_3} \mathcal{L}(y_3, 1_{v_b})$

- 2 Compute $\nabla_{z_3} \hat{R}_b = (\mathbf{JSoft}_{\max})^T \nabla_{y_3} \hat{R}_b$

- 3 Compute $\nabla_{y_2} \hat{R}_b = \mathbf{W}_3^T \nabla_{z_3} \hat{R}_b$

- 4 Remove entry at index 0 of $\nabla_{y_2} \hat{R}_b$ and compute $\nabla_{z_2} \hat{R}_b = \dot{r}(z_2) \odot \nabla_{y_2} \hat{R}_b$

- 5 Compute $\nabla_{y_1} \hat{R}_b = \mathbf{W}_2^T \nabla_{z_2} \hat{R}_b$

- 6 Remove entry at index 0 of $\nabla_{y_1} \hat{R}_b$ and compute $\nabla_{z_1} \hat{R}_b = \dot{r}(z_1) \odot \nabla_{y_1} \hat{R}_b$

Multiclass Classification via FNN: *Training*

Let's now recall **gradient descent** for training of multiclass classifier

GradientDescent() :

- 1: Initiate with some initial values $\{\mathbf{W}_1^{(0)}, \mathbf{W}_2^{(0)}, \mathbf{W}_3^{(0)}\}$ and set a learning rate η
- 2: **while** weights not converged **do**
- 3: **for** $b = 1, \dots, B$ **do**
- 4: $\text{NN.values} \leftarrow \text{ForwardProp}(\mathbf{x}_b, \{\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)}, \mathbf{W}_3^{(t)}\})$
- 5: $\{\mathbf{G}_{1,b}, \mathbf{G}_{2,b}, \mathbf{G}_{3,b}\} \leftarrow \text{BackProp}(\mathbf{x}_b, \mathbf{u}_b, \{\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)}, \mathbf{W}_3^{(t)}\}, \text{NN.values})$
- 6: **end for**
- 7: Update

$$\mathbf{W}_1^{(t+1)} \leftarrow \mathbf{W}_1^{(t)} - \eta \text{ mean}(\mathbf{G}_{1,1}, \dots, \mathbf{G}_{1,B})$$

$$\mathbf{W}_2^{(t+1)} \leftarrow \mathbf{W}_2^{(t)} - \eta \text{ mean}(\mathbf{G}_{2,1}, \dots, \mathbf{G}_{2,B})$$

$$\mathbf{W}_3^{(t+1)} \leftarrow \mathbf{W}_3^{(t)} - \eta \text{ mean}(\mathbf{G}_{3,1}, \dots, \mathbf{G}_{3,B})$$

8: **end while**

We call this form of gradient descent **full-batch**